

Raport Techniczny: Rekrutacja Whitefly

Stanowisko: Python Developer

Jakub Barszczewski

29 kwietnia 2026

Spis treści

1	Implementacja formularzy (Zadanie 1 i 2)	2
1.1	Flask – wariant synchroniczny (Zadanie 1)	2
1.2	Flask – wariant asynchroniczny z Celery i Redis (Zadanie 1)	2
1.3	FastAPI – wariant synchroniczny (Zadanie 2)	2
1.4	FastAPI – wariant asynchroniczny z Celery i Redis (Zadanie 2)	3
2	Analiza Porównawcza Architektur (Zadanie 3)	4
2.1	Metodologia testów (k6.io)	4
2.2	Scenariusz 1: Flask + uWSGI + Nginx	4
2.3	Scenariusz 2: Flask + ASGI (Uvicorn) + Celery + Redis	5
2.4	Scenariusz 3: FastAPI + ASGI + Nginx	6
2.5	Zestawienie wyników (Tabela zbiorcza)	7
3	Ochrona przed botami (Zadanie 4)	9
3.1	Zastosowane rozwiązanie – Honeypot	9
3.2	Alternatywne rozwiązania	9
3.2.1	Rate limiting	9
3.2.2	CSRF Token	9
3.2.3	FingerprintJS	10
3.2.4	CAPTCHA / reCAPTCHA	10
3.2.5	Analiza zachowania użytkownika	10
4	Wnioski końcowe	11

Implementacja formularzy (Zadanie 1 i 2)

Celem zadań 1 i 2 było zbudowanie formularza zbierającego dane użytkownika (imię i nazwisko) w dwóch wariantach przetwarzania – synchronicznym i asynchronicznym – oraz w dwóch frameworkach: Flask i FastAPI. Poniżej opisano każde z czterech podejść z perspektywy architektonicznej.

1.1 Flask – wariant synchroniczny (Zadanie 1)

Klasyczna architektura blokująca oparta na frameworku **Flask** z rozszerzeniem **Flask-SQLAlchemy**. Serwer odbiera żądanie **POST**, waliduje dane, wykonuje zapis do bazy danych **SQLite** i dopiero po zakończeniu wszystkich operacji zwraca odpowiedź użytkownikowi. Cały cykl *request-response* odbywa się sekwencyjnie w obrębie jednego procesu.

Kluczowe elementy implementacji: Walidacja danych odbywa się bezpośrednio w modelu ORM przy użyciu dekoratora `@validates`, który wyrzuca `ValueError` w przypadku wykrycia cyfr w polach imienia lub nazwiska. Obsługa błędów bazy danych realizowana jest przez blok `try/except` z jawnym wywołaniem `db.session.rollback()`. Komunikaty zwrotne przekazywane są użytkownikowi za pomocą mechanizmu *flash messages* i wzorca *Post-Redirect-Get* (przekierowanie **302 Found**).

Ograniczenia: Każdy worker uWSGI jest zajęty przez cały czas trwania operacji I/O (zapis do **SQLite**). Przy dużym natężeniu ruchu prowadzi to do szybkiego wyczerpania puli procesów i kolejkowania żądań na poziomie serwera sieciowego.

1.2 Flask – wariant asynchroniczny z Celery i Redis (Zadanie 1)

Rozszerzenie poprzedniej architektury o system kolejkowania zadań. Serwer Flask przyjmuje dane formularza, tworzy zadanie w brokerze wiadomości **Redis** za pomocą `insert_user_task.delay` i natychmiast zwraca odpowiedź użytkownikowi – bez oczekiwania na faktyczny zapis do bazy. Zapis wykonywany jest przez osobny proces **Celery worker**, działający niezależnie od serwera webowego.

Kluczowe elementy implementacji: Worker Celery posiada własny kontekst aplikacji Flask (`app_context`), niezbędny do korzystania z sesji **SQLAlchemy**. Instancja **Celery** inicjalizowana jest w osobnym module `tasks.py`, co zapobiega cyklicznym importom. Adapter `WsgiToAsgi` z biblioteki `asgiref` pozwala na uruchomienie aplikacji WSGI (Flask) pod serwerem ASGI (Uvicorn).

Kompromis: Architektura ta przedkłada **niezawodność** nad niską latencję. Każde żądanie jest gwarantowanie przyjęte do kolejki, nawet przy ekstremalnym obciążeniu, kosztem wyższego średniego czasu odpowiedzi.

1.3 FastAPI – wariant synchroniczny (Zadanie 2)

Reimplementacja formularza z użyciem frameworka **FastAPI**. Mimo że FastAPI natywnie obsługuje programowanie asynchroniczne, pozwala na definiowanie standardowych,

synchronicznych endpointów. W tym wariancie logika zapisu do bazy odbywa się bezpośrednio w handlerze żądania, analogicznie do podejścia z Flask Sync.

Kluczowe elementy implementacji: Walidacja danych realizowana jest przez model **Pydantic** (`UserCreate`) z dekoratorem `@field_validator`, który zastępuje walidację na poziomie ORM. Sesja bazy danych zarządzana jest przez mechanizm *dependency injection* FastAPI (`Depends(get_db)`), co zapewnia poprawne otwieranie i zamykanie połączeń. Obsługa błędów walidacji (`ValidationError`) realizowana jest przez przechwycenie wyjątku Pydantic i przekierowanie z parametrem `error` w URL.

1.4 FastAPI – wariant asynchroniczny z Celery i Redis (Zadanie 2)

Docelowa architektura łącząca natywną asynchroniczność FastAPI z systemem kolejkowania Celery. Endpoint `/submit` jedynie przyjmuje dane, waliduje je przez Pydantic i deleguje zapis do kolejki Redis – po czym natychmiast zwraca przekierowanie `303 See Other`. FastAPI uruchamiany jest na serwerze **Uvicorn** (natywne ASGI), eliminując potrzebę stosowania adapterów.

Kluczowe różnice względem Flask Async: FastAPI nie wymaga adaptera WSGI→ASGI, co redukuje narzut komunikacyjny. Pydantic zapewnia silniejszą i bardziej ekspresywną walidację niż dekoratory ORM SQLAlchemy. Wzorzec *Post-Redirect-Get* zrealizowany jest przez `RedirectResponse` z kodem 303, a komunikaty przekazywane są przez parametry URL zamiast sesyjnych flash messages.

Analiza Porównawcza Architektur (Zadanie 3)

W tej sekcji analizujemy, jak różne stosy technologiczne radzą sobie z intensywnym ruchem typu *write-heavy* (częste zapisy do bazy danych).

2.1 Metodologia testów (k6.io)

W celu wszechstronnej oceny wydajności, do przeprowadzenia testów wykorzystano narzędzie **k6.io**. Wszystkie testy opierały się na żądaniach typu `POST`, symulujących wysyłkę formularza, bez stosowania przerw (`sleep`), co pozwoliło na zbadanie maksymalnej przepustowości procesowej.

Uwaga dotycząca rzetelności metodologii: W trakcie pierwszej iteracji testów wykryto dwa błędy w skrypcie k6, które fałszowały wyniki. Po pierwsze, parametr `redirects` nie był ustawiony na 0, przez co k6 podążał za przekierowaniami i mierzył łączny czas dwóch żądań. Po drugie, asercja sprawdzała status 200 zamiast 303/302, co powodowało sztucznie zawyżony wskaźnik błędów. Wszystkie poniższe wyniki pochodzą z poprawionego skryptu.

Zastosowano trzy profile obciążenia:

1. **Load Test (Stabilność):** Symulacja umiarkowanego ruchu (**40 VUs**). Składa się z minuty narastania obciążenia, 3-minutowej fazy stabilnej (`plateau`) oraz wygaszenia.
 - *Cel:* Sprawdzenie zachowania systemu przy stałym, oczekiwanym obciążeniu.
2. **Stress Test (Granica wydajności):** Agresywne zwiększanie liczby użytkowników do poziomu **100 VUs**.
 - *Cel:* Znalezienie punktu, w którym architektura synchroniczna zaczyna generować znaczące opóźnienia i kolejkovanie żądań.
3. **Spike Test (Odporność na skoki):** Nagły, sekundowy skok obciążenia do **150 VUs**.
 - *Cel:* Weryfikacja odporności serwera na nieoczekiwane uderzenia ruchu (np. efekt „viralowy”) i sprawdzenie limitów kolejki (*listen backlog*) serwera uWSGI-/Nginx.

Uwaga dotycząca rzetelności danych: W celu izolacji wydajności logicznej serwera od ograniczeń przepustowości sieciowej, widok HTML został ograniczony do wyświetlania 10 ostatnich rekordów (`limit(10)`). Pozwoliło to uniknąć liniowego wzrostu wagi odpowiedzi wraz z rozmiarem bazy danych i skupić się na wydajności samej architektury.

2.2 Scenariusz 1: Flask + uWSGI + Nginx

Pierwszy scenariusz reprezentuje klasyczną architekturę synchroniczną. Serwer aplikacyjny uWSGI zarządza pulą procesów (`workerów`), z których każdy obsługuje dokładnie jedno żądanie w danym czasie. Jest to model blokujący, w którym proces pozostaje zajęty przez cały cykl życia żądania (od odebrania danych, przez zapis do bazy SQLite, aż po wyrenderowanie odpowiedzi).

Analiza wyników pomiarów: Wyniki testów jednoznacznie wskazują na sztywne limity wydajnościowe tej architektury:

Typ Testu	RPS	Avg Latency	p(95)	Failure Rate
Load Test (40 VUs)	108.31	308.36 ms	397.38 ms	0.00%
Stress Test (100 VUs)	103.88	543.95 ms	1.08 s	0.00%
Spike Test (150 VUs)	1228.77*	617.72 ms**	1.13 s**	91.93%

Tabela 1: Wyniki wydajności dla architektury synchronicznej uWSGI. (*wartość próbowa, **dla żądań udanych)

Kluczowe obserwacje i interpretacja:

- **Saturacja procesów (Stress):** Przy zwiększeniu liczby użytkowników z 40 do 100, przepustowość (RPS) nie wzrosła, a wręcz nieznacznie spadła (z 108 do 103). Świadczy to o pełnym wysyceniu dostępnych workerów uWSGI. Każdy dodatkowy użytkownik musiał oczekiwać w kolejce, co spowodowało wzrost opóźnienia $p(95)$ powyżej sekundy.
- **Katastrofalna awaria (Spike):** Nagły skok do 150 użytkowników doprowadził do odrzucenia aż **91.93% żądań**. Jest to klasyczny przykład przepełnienia kolejki połączeń (*listen backlog*). Synchroniczna natura workerów sprawiła, że nie byli oni w stanie zwalniać zasobów wystarczająco szybko, by przyjąć nowe pakiety TCP, co zmusiło system operacyjny/Nginx do zamykania połączeń.
- **Efektywność zapisu:** Mimo błędów w Spike Teście, żądania które „przeszły” zachowały względnie stabilny czas odpowiedzi (avg 617 ms), co potwierdza, że wąskim gardłem nie jest sama logika aplikacji, lecz mechanizm zarządzania połączeniami przychodzącymi.

Wnioski techniczne: Architektura synchroniczna uWSGI charakteryzuje się wysoką przewidywalnością i stabilnością, ale tylko do punktu saturacji workerów. Brak mechanizmów asynchronicznych sprawia, że system jest całkowicie nieodporny na skokowe zmiany natężenia ruchu. Wyniki te stanowią bezpośrednie uzasadnienie dla poszukiwania alternatywnych rozwiązań (Scenariusz 2 i 3), które potrafią oddzielić warstwę obsługi połączeń od warstwy logiki biznesowej.

2.3 Scenariusz 2: Flask + ASGI (Uvicorn) + Celery + Redis

W tym scenariuszu wdrożono architekturę asynchroniczną z wykorzystaniem systemu kolejkowego. Serwer **Uvicorn** obsługuje połączenia przychodzące, a czasochłonne operacje zapisu do bazy danych są delegowane do workera **Celery** za pośrednictwem brokera wiadomości **Redis**.

Analiza wyników pomiarów: Głównym atutem tej konfiguracji okazała się całkowita odporność na błędy, niezależnie od charakterystyki ruchu.

Typ Testu	RPS	Avg Latency	p(95)	Failure Rate
Load Test (40 VUs)	69.97	485.73 ms	1.28 s	0.00%
Stress Test (100 VUs)	83.75	679.32 ms	2.48 s	0.00%
Spike Test (150 VUs)	58.68	1.36 s	3.22 s	0.00%

Tabela 2: Wyniki wydajności dla architektury Flask z systemem kolejkowym.

Kluczowe obserwacje i interpretacja:

- **Pełna niezawodność:** Architektura uzyskała **100% skuteczności** we wszystkich próbach. W porównaniu do Scenariusza 1, gdzie Spike Test zakończył się awarią 87% żądań, wprowadzenie Redisa jako bufora pozwoliło na bezbłędne przyjęcie każdego zapytania.
- **Charakterystyka latencji:** Średnie czasy odpowiedzi oscylują w granicach 485–1360 ms. Wyższe wartości w porównaniu do FastAPI wynikają z narzutu komunikacyjnego między Flaskiem, Redisem a Celery oraz zastosowanego wzorca oczekiwania na wynik zadania.
- **Prawidłowa obsługa przekierowań:** Testy potwierdziły poprawność implementacji mechanizmu *Post-Redirect-Get* (status **303 See Other**), co pozwala na zachowanie spójności stanu aplikacji po stronie klienta przy jednoczesnym asynchronicznym przetwarzaniu danych.
- **Stabilność pod obciążeniem (Stress):** Przy 100 VUs system utrzymał stabilny przepływ blisko 84 żądań na sekundę, co świadczy o dobrej skalowalności puli workerów Celery.

Wnioski techniczne: Architektura oparta na Celery i Redis jest rozwiązaniem typu *enterprise-grade*, przedkładającym stabilność i gwarancję zapisu nad ekstremalnie niską latencję. Jest to idealny wybór dla systemów, gdzie priorytetem jest obsłużenie każdego klienta nawet kosztem dłuższego czasu oczekiwania. W zestawieniu ze Scenariuszem 1, jest to skokowy wzrost jakości infrastruktury przy zachowaniu tego samego frameworka aplikacyjnego (Flask).

2.4 Scenariusz 3: FastAPI + ASGI + Nginx

Trzecia architektura opiera się na frameworku **FastAPI**, który natywnie obsługuje asynchroniczność w standardzie ASGI. W przeciwieństwie do poprzednich scenariuszy, system ten został zaprojektowany do obsługi dużej liczby współbieżnych połączeń bez narzutu związanego z adapterami WSGI czy zewnętrznymi systemami kolejkowymi dla prostych operacji I/O.

Analiza wyników pomiarów: Nowe testy wykazały, że FastAPI jest najbardziej wydajną z badanych architektur, oferując najniższe opóźnienia i najwyższą stabilność:

Typ Testu	RPS	Avg Latency	p(95)	Failure Rate
Load Test (40 VUs)	78.92	106.50 ms	58.72 ms	0.64%
Stress Test (100 VUs)	79.11	400.80 ms	156.29 ms	0.02%
Spike Test (150 VUs)	69.22	335.30 ms	86.70 ms	0.66%

Tabela 3: Wyniki wydajności dla natywnej architektury FastAPI + ASGI.

Kluczowe obserwacje i interpretacja:

- **Ekstremalnie niskie opóźnienia:** W teście obciążeniowym (Load) percentyl $p(95)$ wyniósł zaledwie **58.72 ms**. Jest to wynik niemal 4-krotnie lepszy niż w przypadku synchronicznego Flaska (216 ms) i o rząd wielkości lepszy niż w Scenariuszu 2 z Celery.
- **Efektywność sieciowa:** Dzięki przesyłaniu danych w formacie zoptymalizowanym oraz sprawnemu zarządzaniu odpowiedziami 303 (See Other), zużycie pasma sieciowego spadło drastycznie. W teście Load przesłano jedynie **4.5 MB** danych (w porównaniu do 93 MB we Flasku), co minimalizuje ryzyko zatorów na warstwie sieciowej.
- **Odporność na skoki (Spike):** W teście Spike (150 VUs) FastAPI utrzymało $p(95)$ na poziomie poniżej **90 ms**. Jest to kluczowy wniosek – architektura ta radzi sobie z nagłym uderzeniem ruchu lepiej niż systemy kolejkowe, nie wymagając przy tym dodatkowej infrastruktury (Redis).
- **Interpretacja błędów:** Śladowy odsetek niepowodzeń (poniżej 1%) wynikał głównie z pojedynczych przekroczeń limitów czasu przy maksymalnym nasyceniu procesora, co przy braku błędów krytycznych (5xx) potwierdza wysoką niezawodność stosu.

Wnioski techniczne: Scenariusz 3 udowodnił, że FastAPI w połączeniu z serwerem Uvicorn stanowi optymalny wybór dla aplikacji typu *write-heavy*. Natywne wsparcie dla asynchroniczności pozwala na efektywne wykorzystanie zasobów serwera bez blokowania procesów roboczych podczas zapisu do SQLite. Jest to rozwiązanie łączące najwyższą wydajność (niskie $p(95)$) z prostotą architektury (brak konieczności utrzymywania workerów Celery).

2.5 Zestawienie wyników (Tabela zbiorcza)

Poniższa tabela stanowi kluczowe porównanie wszystkich badanych konfiguracji pod kątem wydajności procesowej (Load Test) oraz odporności na nagłe skoki ruchu (Spike Test).

Architektura	RPS (Load)	Avg Latency	p(95)	Errors (Spike)
Flask + uWSGI (Sync)	108.31	308.36 ms	397.38 ms	91.93%
Flask + Celery (Async)	69.97	485.73 ms	1.28 s	0.00%
FastAPI + ASGI (Native)	78.92	106.50 ms	58.72 ms	0.66%

Tabela 4: Porównanie wydajności badanych stosów technologicznych.

Wnioski z zestawienia:

- **Najszybsza reakcja:** FastAPI deklasuje konkurencję pod kątem latencji. Wynik $p(95) = 58.72$ ms jest prawie 7-krotnie lepszy niż w klasycznym Flasku.
- **Najwyższa stabilność:** Architektura z Celery i Redisem jako jedyna osiągnęła perfekcyjny wynik 0% błędów przy ekstremalnym skoku obciążenia, co czyni ją najbezpieczniejszym wyborem dla systemów krytycznych.
- **Punkt krytyczny:** Tradycyjny model synchroniczny (uWSGI) mimo przyzwoitej przepustowości (RPS), jest całkowicie nieodporny na ataki typu Spike, co dyskwalifikuje go w zastosowaniach narażonych na nagłe skoki ruchu.

Ochrona przed botami (Zadanie 4)

3.1 Zastosowane rozwiązanie – Honeypot

W celu ograniczenia fałszywych rejestracji generowanych przez boty zaimplementowano mechanizm **honeypot**. Polega on na dodaniu do formularza HTML ukrytego pola tekstowego (w tym przypadku pola `website`), które jest niewidoczne dla prawdziwego użytkownika dzięki zastosowaniu stylu `display:none`, atrybutu `tabindex=1` oraz `autocomplete="off"`.

```
<input type="text" name="website"
       style="display:none"
       tabindex="1"
       autocomplete="off">
```

Po stronie serwera endpoint `/submit` sprawdza, czy pole `website` zostało wypełnione. Boty automatycznie wypełniają wszystkie pola formularza, natomiast prawdziwy użytkownik nie widzi ukrytego pola i pozostawia je puste. W przypadku wykrycia wypełnionego pola honeypot serwer zwraca odpowiedź `303 See Other` – identyczną jak przy poprawnym zgłoszeniu – nie informując bota o zablokowaniu żądania.

```
if website:
    return RedirectResponse(
        url="/?success=Task submitted to background queue",
        status_code=303
    )
```

Skuteczność rozwiązania została zweryfikowana za pomocą testów obciążeniowych przeprowadzonych narzędziem `k6`. Symulowano atak botów wypełniających pole honeypot wartością `http://spam.com` przy 100 wirtualnych użytkownikach. Po zakończeniu testu (8932 żądania) wykonano zapytanie do bazy danych:

```
SELECT COUNT(*) FROM users WHERE name='Bot';
```

Wynik 0 potwierdził, że żadne zgłoszenie od bota nie zostało zapisane w bazie danych pomimo wysokiego natężenia ruchu.

3.2 Alternatywne rozwiązania

3.2.1 Rate limiting

Ograniczenie liczby żądań z jednego adresu IP w określonym przedziale czasowym. W ekosystemie FastAPI można je zrealizować za pomocą biblioteki `slowapi`. Metoda skutecznie blokuje ataki volumetryczne, jednak nie chroni przed botami operującymi na dużej puli adresów IP (np. sieci botnet).

3.2.2 CSRF Token

Mechanizm polegający na osadzeniu w formularzu jednorazowego tokenu sesyjnego, który musi zostać odesłany wraz z żądaniem POST. Bot wysyłający żądanie bez uprzedniego pobrania strony nie dysponuje aktualnym tokenem i zostaje odrzucony. W FastAPI można to zrealizować przy użyciu biblioteki `fastapi-csrf-protect`.

3.2.3 FingerprintJS

Biblioteka JavaScript zbierająca po stronie klienta sygnały przeglądarki (canvas fingerprint, czcionki, WebGL, strefa czasowa itp.) w celu wygenerowania unikalnego identyfikatora urządzenia. Serwer może wykryć, że ten sam identyfikator wysyła dużą liczbę żądań w krótkim czasie, i zablokować go. Rozwiązanie skuteczne przeciwko zaawansowanym botom, jednak wymaga integracji z zewnętrznym serwisem lub własną logiką po stronie backendu.

3.2.4 CAPTCHA / reCAPTCHA

Weryfikacja oparta na wyzwaniach wizualnych lub behawioralnych (Google reCAPTCHA v2/v3, hCaptcha). Wersja v3 działa w tle i przyznaje każdemu żądaniu ocenę ryzyka od 0 do 1 bez angażowania użytkownika. Jest to jedno z najbardziej powszechnych rozwiązań, jednak wiąże się z zależnością od zewnętrznego dostawcy oraz potencjalnymi problemami z prywatnością użytkowników.

3.2.5 Analiza zachowania użytkownika

Monitorowanie wzorców interakcji z formularzem – czasu wypełniania pól, ruchu myszą, kolejności kliknięć. Boty zazwyczaj wypełniają formularze natychmiastowo i w deterministyczny sposób, co odróżnia je od zachowania człowieka. Implementacja wymaga dodatkowej logiki JavaScript oraz analizy po stronie serwera.

Wnioski końcowe

Przeprowadzone zadania pozwoliły na praktyczne porównanie trzech fundamentalnie różnych modeli architektury webowej w Pythonie. Każde z rozwiązań reprezentuje inny punkt na spektrum kompromisów między prostotą, wydajnością i niezawodnością.

Flask + uWSGI sprawdza się jako szybkie i proste rozwiązanie dla aplikacji o przewidywalnym, niskim ruchu. Jego synchroniczna natura jest jednocześnie jego największą zaletą (prostota debugowania, przewidywalność) i wadą (brak odporności na nagłe skoki ruchu). Model ten powinien być traktowany jako punkt wyjścia, nie jako architektura docelowa dla systemów produkcyjnych.

Flask + Celery + Redis wprowadza warstwę niezawodności, której brakuje w modelu synchronicznym. Gwarancja przyjęcia każdego żądania do kolejki czyni tę architekturę optymalną dla systemów, gdzie utrata danych jest niedopuszczalna (płatności, rejestracje, powiadomienia). Kosztem jest dodatkowa infrastruktura (broker Redis, procesy workerów) oraz wyższa latencja odczuwalna przez użytkownika.

FastAPI + ASGI okazał się najlepszym wyborem dla scenariusza testowego – oferuje najniższe opóźnienia przy zachowaniu dobrej odporności na obciążenie. Natywna asynchroniczność eliminuje potrzebę zewnętrznych adapterów, a ekosystem Pydantic zapewnia elegancką walidację danych. Jest to architektura rekomendowana dla nowych projektów, gdzie wydajność i nowoczesność stosu technologicznego mają znaczenie.

W zakresie ochrony przed botami, zastosowany mechanizm honeypot potwierdził swoją skuteczność w testach – żadne z 8932 spreparowanych żądań nie zostało zapisane do bazy. Jest to rozwiązanie o zerowym koszcie wydajnościowym i minimalnej złożoności implementacyjnej, stanowiące dobrą pierwszą linię obrony przed prostymi botami. Dla środowisk produkcyjnych zalecane jest uzupełnienie go o rate limiting oraz tokeny CSRF.